

Eijen Hash Function

Keting Jia* Qingliang Hou Guoxiao Liu Zhuoqing Peng
Yantian Shen Xiaoen Lin Yukun Liu

*Corresponding submitter: Keting Jia
Tsinghua University, Beijing, P.R.China
`ktjia@tsinghua.edu.cn`

Tsinghua University, Beijing, P.R.China
Zhongguancun Laboratory, Beijing, P.R.China
Shandong University, Qingdao, P.R.China
State Key Laboratory of Cryptography and
Digital Economy Security, P.R.China

Contents

Revision Note	3
1 Introduction	4
2 Specification of Eijen	4
2.1 Notations	4
2.2 Recommended Instances and Parameter Sets	5
2.3 Sponge-F Mode	5
2.4 Specification of Permutation	7
2.4.1 MixQuad Layer	8
2.4.2 PosPerm Layer	10
2.4.3 AMix4 Layer	10
3 Security Claim	11
4 Security Analysis	11
4.1 Differential Cryptanalysis	11
4.2 Differential-Linear Cryptanalysis	12
4.3 Integral Cryptanalysis	13
4.4 Preimage Cryptanalysis	14
4.5 (Quantum) Collision Cryptanalysis	14
4.6 Other Cryptanalysis	14
5 Design Rationale	16
5.1 Sponge-F Structure	16
5.2 Design Choices for Permutation	17
6 Features	18
7 Implementation and Evaluation	19
7.1 Software Implementations	19
7.2 Evaluation Methodology	20
7.3 Performance Results	20
8 Randomness Tests	21

A	Round Constants	24
B	Test Vectors	25
B.1	Test Vectors for Eijen-256	25
B.2	Test Vectors for Eijen-384	28
B.3	Test Vectors for Eijen-512	29
B.4	Test Vectors for Eijen-768	30
B.5	Test Vectors for Eijen-1024	31

Revision Note

This edition is the normative little-endian, pad_{10*1} revision of the Eijen specification. It first adopts the little-endian message and word mapping used in the submitted Eijen document while replacing the ambiguous rate padding with a complete pad_{10*1} rule. Then, the final capacity constant changes from 1_c to D_h , whose final word is $2^{63} + h$. The constant is injected before the permutation and included in capacity feed-forward. These changes are intended to prevent the identified non-random relation between Eijen instances with different digest lengths. We also correct the padding implementation for bit-aligned messages and regenerate the affected materials. The permutation, parameter sets, zero IV, and digest extraction positions are unchanged. All test vectors are regenerated; digests are incompatible with the previous Eijen.

1 Introduction

In this document, we present the Eijen hash-function family. The design utilizes a feed-forward Sponge mode [1, 2] instantiated within a 2048-bit Add-Rotate-Xor (ARX) permutation, yielding a family of fixed-output hash algorithms Eijen- h , where h is the size of digest and $h \in \{256, 384, 512, 768, 1024\}$. The design has two goals. First, at the mode level, it aims to inherit the feature of feed-forward Sponge (Sponge-F) mode to break invertibility, and exact security claims against collision, preimage, and second-preimage attack in both classical and quantum settings. Second, at the permutation level, we use a 4×8 array of 64-bit words and word-based mixing operations to obtain fast diffusion and efficient software and hardware implementations.

2 Specification of Eijen

2.1 Notations

To avoid notational ambiguity, the following notations are adopted throughout this document:

- b denotes the state size in bits;
- r denotes the rate, representing the bit length of the message block;
- c denotes the capacity, which determines the security level of the algorithm;
- h denotes the bit length of the output hash digest;
- r' denotes the width squeezed from the capacity part;
- $\text{right}_i(X)$ denotes the rightmost i bits of X ;
- H_h denotes the fixed-output hash algorithm with a digest length of h bits;
- Π denotes the full 2048-bit permutation;
- $\mathcal{R}^{(i)}$ denotes the i -th round function of Π ;
- MixQuad denotes the column-wise word-mixing layer;
- PosPerm denotes the word position permutation layer;
- AMix4 denotes the almost MixColumn layer.

Normative Serialization Convention. All algorithm descriptions in this document use little-endian order. Message bits are consumed LSB-first within each byte: bit zero is the least significant bit of the first message byte. Consecutive groups of eight message or state bytes are interpreted as little-endian 64-bit words, with $A[0]$ containing the first eight state bytes. Digest words are emitted in state-word order, with each word serialized little-endian. If the declared message length is not byte aligned, the valid bits of the final input byte are its low-order bits; unused high-order bits are ignored.

2.2 Recommended Instances and Parameter Sets

This document specifies a family of hash algorithms **Eijen**, which is based on the feed-forward Sponge-F construction introduced in [1]. **Eijen** includes five variants **Eijen- h** instantiated within a common 2048-bit ARX permutation, while different output digest size $h \in \{256, 384, 512, 768, 1024\}$. Table 1 summaries the five recommended instances and their corresponding parameter sets.

These parameter sets are carefully chosen to satisfy the $h \leq c$ constraint mandated for the fixed-output versions, while simultaneously maximizing the absorption rate. Specifically, all configurations adhere to a unified capacity formula $c = h + 64$, where $64 = \lceil \log_2 \ell_{\max} \rceil$ corresponds to the maximum number of message blocks $\ell_{\max} = 2^{64} - 1$. This design ensures that the second-preimage security, bounded by $\min\{h, c - \log_2 \ell\}$, maintains a full h -bit security level even at the maximum message length of $2^{64} - 1$.

Table 1: Recommended Instances of **Eijen**

Function	b	r	c	r'	h
Eijen-256	2048	1728	320	256	256
Eijen-384	2048	1600	448	384	384
Eijen-512	2048	1472	576	512	512
Eijen-768	2048	1216	832	768	768
Eijen-1024	2048	960	1088	1024	1024

2.3 Sponge-F Mode

Let $m = |M|$. **Eijen** uses the multi-rate padding rule pad_{10^*1} specified in FIPS 202, Algorithm 9 [3]:

$$j = (-m - 2) \bmod r, \quad 0 \leq j < r,$$

$$\text{pad}_{10^*1,r}(M) = M \| 1 \| 0^j \| 1.$$

The SHA-3 hash-function domain suffix 01 is not appended; only the pad_{10^*1} rule is adopted. The padded string is divided into $\ell = (m + j + 2)/r$ blocks:

$$\text{pad}_{10^*1,r}(M) = M[0] \| M[1] \| \cdots \| M[\ell - 1], \quad M[i] \in \{0, 1\}^r.$$

Under the normative little-endian convention, the first padding bit occupies the next LSB-first bit position, and the final padding bit is the most significant bit of the last rate byte. For a byte-aligned message, these two markers are represented by 01 in the first unused byte and 80 in the final rate byte; if they share one byte, that byte is 81. If $m \equiv r - 1 \pmod{r}$, then $j = r - 1$ and the padding spans two blocks. For the 24-bit message “abc” in Eijen-256, the first padded 64-bit word is 0000000001636261, and the final rate word is 8000000000000000.

The message absorbing phase and the digest squeezing phase are illustrated in Figure 1. Let $X_i \in \{0, 1\}^b$ denote the internal state. The detailed absorbing and squeezing phases for Eijen hash function are shown as follows:

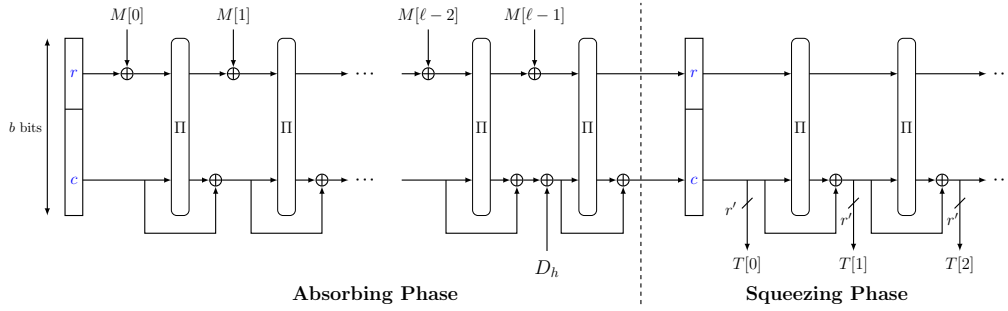


Figure 1: Sponge-F Mode

Initialization. The initial state is defined as

$$X_0 = 0^r \| IV,$$

where IV is defaulted to 0^c in Eijen hash function.

Absorbing Phase. For the message blocks before the last block, we define:

$$\begin{cases} S_{i-1} &= X_{i-1} \oplus (M[i-1] \| 0^c), \\ X_i &= \Pi(S_{i-1}) \oplus (0^r \| \text{right}_c(S_{i-1})), \end{cases}$$

for $1 \leq i \leq \ell - 1$. For the last message block, an instance-specific constant D_h is XORed into the capacity part:

$$\begin{cases} S_{\ell-1} &= X_{\ell-1} \oplus (M[\ell-1] \| D_h), \\ X_\ell &= \Pi(S_{\ell-1}) \oplus (0^r \| \text{right}_c(S_{\ell-1})) \end{cases}$$

All words of D_h are zero except its final 64-bit word, whose numeric value is

$$\text{tag}(h) = 2^{63} + h = \text{0x8000000000000000 OR } h.$$

The low 16 bits encode the supported digest length h as an unsigned integer; bits 16–62 are zero and bit 63 retains the final-block marker. The implementation XORs this value into absolute state word $A[31]$. This word lies in the capacity of all five instances, regardless of the

rate. For $h = 256, 384, 512, 768, 1024$, the constants are respectively 80000000000000100, 80000000000000180, 80000000000000200, 80000000000000300, and 80000000000000400. For example, D_{512} ends in the bytes 00 02 00 00 00 00 00 80. The feed-forward value is $\text{right}_c(S_{\ell-1})$, including the injected D_h . The order is final rate absorption, tag injection, capacity snapshot, permutation, and capacity feed-forward. Applying the tag only after the permutation is not equivalent and is not permitted. The rate-side terminal padding bit and the capacity-side tag remain independent. No message-length field is appended or injected in this revision.

Fixed-Output Extraction. For the fixed-output hash algorithm H_h , the digest is extracted from the capacity part of the final state:

$$H_h(M) = \text{right}_h(X_\ell). \quad (1)$$

Given that the rightmost c bits of the state correspond precisely to the capacity, this operation is equivalent to applying an h -bit truncation to the final capacity.

Full-Capacity Extraction. When $h = c$, the construction degenerates to outputting the entire c bits directly from the final capacity, yielding:

$$H_c(M) = \text{right}_c(X_\ell). \quad (2)$$

In the currently recommended parameter sets, all configurations satisfy the condition $c = h + 64 > h$. Consequently, the output digest invariably results from an h -bit truncation of the final capacity, rather than a full-capacity extraction.

Optional XOF Extension. If an extendable-output function (XOF) interface is required, one can repeatedly extract r' bits from the capacity part following the standard Sponge-F methodology. This extracted value is then fed forward into the rightmost r' bit positions of the state for the subsequent round. However, as this document mainly focuses on fixed-output hashing, it omits a dedicated XOF specification for this concrete instantiation.

2.4 Specification of Permutation

The core component of Eijen is a permutation $\Pi : \{0, 1\}^{2048} \rightarrow \{0, 1\}^{2048}$ operating on a large-scale state of 2048 bits. The 2048-bit input string is rearranged into a $4 \times 8 \times 64$ three-dimensional array A . The state bit at index (i, j, k) within this array is denoted as $A[i][j][k]$, where $0 \leq i < 4$, $0 \leq j < 8$, and $0 \leq k < 64$. Let $A[i + 4 \cdot j] \in \mathbb{Z}_{2^{64}}$ represent the 64-bit word at index (i, j) . Furthermore, let A_j denote the four 64-bit words comprising the j -th column, defined as:

$$A_j = (A[4 \cdot j], A[4 \cdot j + 1], A[4 \cdot j + 2], A[4 \cdot j + 3])^T, \quad j = 0, 1, \dots, 7.$$

Each round function of the permutation consists of three distinct operations: the non-linear layer **MixQuad** (MQ), the word position permutation layer **PosPerm** (P), and the almost MixColumn layer **AMix4** (AM). The i -th round transformation $\mathcal{R}^{(i)}$ is defined as follows:

$$A^{(i)} \xrightarrow{\text{MixQuad}} Z^{(i)} \xrightarrow{\text{PosPerm}} U^{(i)} \xrightarrow{\text{AMix4}} W^{(i)} \xrightarrow{\text{PosPerm}} A^{(i+1)}$$

The permutation iterates for a total of 16 rounds, that is, $\Pi = \mathcal{R}^{(15)} \circ \mathcal{R}^{(14)} \circ \dots \circ \mathcal{R}^{(0)}$.

2.4.1 MixQuad Layer

The **MixQuad** operation is applied to each column of the internal state matrix, taking four 64-bit words (a, b, c, d) as input. For instance, when applied to the j -th column of the matrix, we have $(a, b, c, d) = (A[4 \cdot j], A[4 \cdot j + 1], A[4 \cdot j + 2], A[4 \cdot j + 3])$. Furthermore, we define the following left-rotation function on 64-bit words:

$$\text{ROTL}_{64}(x, n) = (x \ll n) \vee (x \gg (64 - n)).$$

Given the rotation parameters $\pi = (\pi_0, \pi_1, \pi_2, \pi_3, \pi_4, \pi_5)$ and the round constant **ci**, the **MixQuad** operation is illustrated in Figure 2 and is formally defined as follows:

$$\text{MQ}_{\pi, \text{ci}}(a, b, c, d) : \begin{cases} a \leftarrow a \boxplus b, & d \leftarrow d \oplus a, & d \leftarrow \text{ROTL}_{64}(d, \pi_0), & a \leftarrow \text{ROTL}_{64}(a, \pi_4) \oplus \text{ci}, \\ c \leftarrow c \boxplus d, & b \leftarrow b \oplus c, & b \leftarrow \text{ROTL}_{64}(b, \pi_1), & c \leftarrow \text{ROTL}_{64}(c, \pi_5), \\ a \leftarrow a \boxplus b, & d \leftarrow d \oplus a, & d \leftarrow \text{ROTL}_{64}(d, \pi_2), & \\ c \leftarrow c \boxplus d, & b \leftarrow b \oplus c, & b \leftarrow \text{ROTL}_{64}(b, \pi_3). & \end{cases}$$

Here, all additions $\boxplus$ are performed modulo 2^{64} . Compared to the ChaCha round function,

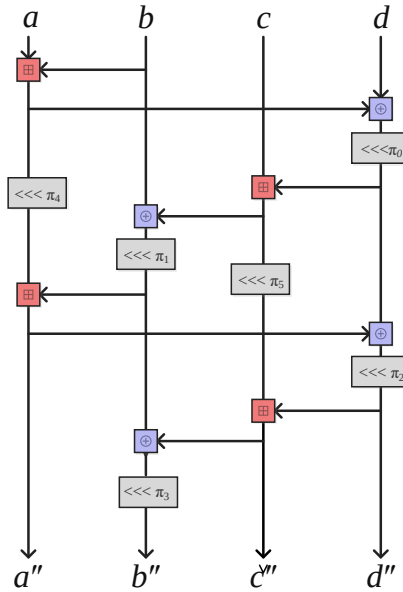


Figure 2: MixQuad Layer

the proposed design introduces an additional operation $a \leftarrow \text{ROTL}_{64}(a, \pi_4) \oplus \text{ci}$ in the first

quarter-round, as well as an operation $c \leftarrow \text{ROTL}_{64}(c, \pi_5)$ in the second quarter-round. The design rationale for these modifications is as follows:

- $\text{ROTL}_{64}(a, \pi_4) \oplus \text{ci}$: The additional rotation shifts the most significant bit (MSB) difference away from the MSB position, followed by the injection of a round constant. This modification breaks the deterministic differential propagation of words a and d at the highest bits during the inverse **MixQuad** operation (reducing the propagation probability from 1 in the original design to $\leq 2^{-1}$). Furthermore, the injection of the round constant effectively eliminates the vulnerability to slide attack.
- $\text{ROTL}_{64}(c, \pi_5)$: This rotation increases the diffusion depth of word c , thereby reducing the optimal probability of Δc in the forward differential (from 2^{-2} down to 2^{-3}).

For columns 0, 1, 2, and 3 (the left half) of the internal state matrix, the proposed design employs the following rotation parameters:

$$p_0 = (36, 54, 45, 44, 18, 9),$$

For columns 4, 5, 6, 7 (the right half) of the internal state matrix, the proposed design employs the following rotation parameters:

$$p_1 = (56, 46, 54, 34, 19, 51).$$

Therefore,

$$Z_j = \begin{cases} \text{MQ}_{p_0, \text{ci}_0^{(t)}}(A_j), & 0 \leq j < 4, \\ \text{MQ}_{p_1, \text{ci}_1^{(t)}}(A_j), & 4 \leq j < 8, \end{cases}$$

where $\text{ci}_0^{(t)}$ and $\text{ci}_1^{(t)}$ denote the round constants for the left and right halves in the t -th round, respectively.

Round Constants. Each round employs two 64-bit constants, which are injected into the **MixQuad** layer for the left and right halves, respectively. The entire set of 32 constants is derived from the hexadecimal expansion of the fractional part of π , extracted sequentially in 64-bit words:

$$\begin{aligned} \text{ci}_0^{(0)} &= \text{0x243F6A8885A308D3}, & \text{ci}_1^{(0)} &= \text{0x13198A2E03707344}, \\ \text{ci}_0^{(1)} &= \text{0xA4093822299F31D0}, & \text{ci}_1^{(1)} &= \text{0x082EFA98EC4E6C89}, \\ & \dots & & \\ \text{ci}_0^{(15)} &= \text{0xB4CC5C341141E8CE}, & \text{ci}_1^{(15)} &= \text{0xA15486AF7C72E993}. \end{aligned}$$

The complete list is provided in the Appendix A. All constants are publicly verifiable, ensuring full transparency and auditability of their generation process. Any interested party can independently reproduce this procedure to verify its correctness.

2.4.2 PosPerm Layer

Let Z denote the input to the word position permutation layer **PosPerm**, and let U be its corresponding output. The specific permutation mapping is defined as follows:

$$U[l] = Z[\sigma(l)], \quad l = 0, 1, \dots, 31,$$

where

$$\sigma = [22, 0, 29, 19, 26, 4, 1, 23, 30, 8, 5, 27, 2, 12, 9, 31, \\ 6, 16, 13, 3, 10, 20, 17, 7, 14, 24, 21, 11, 18, 28, 25, 15].$$

Equivalently, this can be expressed in matrix form as:

$$Y = \begin{bmatrix} 22 & 26 & 30 & 2 & 6 & 10 & 14 & 18 \\ 0 & 4 & 8 & 12 & 16 & 20 & 24 & 28 \\ 29 & 1 & 5 & 9 & 13 & 17 & 21 & 25 \\ 19 & 23 & 27 & 31 & 3 & 7 & 11 & 15 \end{bmatrix}.$$

2.4.3 AMix4 Layer

Let U be the input to the almost MixColumn layer **AMix4**. This layer performs a left-multiplication of each input column by a mixing matrix to produce the output W . Let the j -th column of U be represented as:

$$U_j = (U[4 \cdot j], U[4 \cdot j + 1], U[4 \cdot j + 2], U[4 \cdot j + 3])^T.$$

Define the column-mixing matrix as

$$L = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}.$$

The output column W_j is computed as follows:

$$W_j = L \cdot U_j,$$

where the multiplication is performed over the word-level field $\mathbb{F}_2^{64}$. Consequently, the operation is implemented via XOR as:

$$\begin{cases} W[4j] &= U[4j+1] \oplus U[4j+2] \oplus U[4j+3], \\ W[4j+1] &= U[4j] \oplus U[4j+2] \oplus U[4j+3], \\ W[4j+2] &= U[4j] \oplus U[4j+1] \oplus U[4j+3], \\ W[4j+3] &= U[4j] \oplus U[4j+1] \oplus U[4j+2]. \end{cases}$$

3 Security Claim

In this design, the newly defined 2048-bit permutation Π is combined with the Sponge-F paradigm to construct a hash algorithm. The adopted Sponge-F mode, with a digest length of h , features provable security against collision, preimage, and second-preimage attacks. Specifically, it provides approximately $h/2$ bits of collision resistance and approximately h bits of preimage resistance. For a challenge message with ℓ blocks after padding, it offers approximately $\min\{h, c - \log_2 \ell\}$ bits of second-preimage resistance.

Security Statement for the Current Instantiation. For the Eijen instantiation specified in this document, we claim that the design achieves the aforementioned security goals. Specifically, the 16-round permutation Π provides sufficient resistance against specialized structural attacks, differential cryptanalysis, linear cryptanalysis, meet-in-the-middle attacks, and other cryptanalytic methods. Accordingly, the security strength claims for Eijen are summarized in Table 2.

Table 2: Security Strength of Eijen

Function	h	Collision Security	Preimage Security	Second Preimage Security
Eijen-256	256	128	256	256
Eijen-384	384	192	384	384
Eijen-512	512	256	512	512
Eijen-768	768	384	768	768
Eijen-1024	1024	512	1024	1024

Scope of the Claims. This document primarily focuses on the security goals of the fixed-output hash algorithm family regarding collision, preimage, and second-preimage resistance. In addition, the assessment covers partial-collision attacks, semi-free-start collision attacks, free-start collision attacks, multi-collision attacks, length-extension attacks, and attacks targeting any fixed subset of the output bits. It does not provide a specific indistinguishability proof for the current instantiation, nor does it derive security conclusions for the extendable-output function (XOF) in this section.

4 Security Analysis

4.1 Differential Cryptanalysis

Differential cryptanalysis is one of the most powerful techniques for analyzing symmetric ciphers. The first step for the evaluation of the security against differential attack is to find differential trails with probability much higher than for a random function. Since the operation of differences is linear, the differences propagation through linear operations is

non-probabilistic and deterministic. While the propagation of differences for non-linear operations are probabilistic.

In the underlying permutation Π of Eijen, the only nonlinear layer employs an improved quarter-round (QR) function of ChaCha, which is based on modular additions. In the extensive literature, various automatic methods has been proposed to model the probabilistic differences propagation through modular additions [4, 5]. However, the efficiency of automatic model is highly limited by the number of variables and constraints. In this paper, we use the SAT-based automatic tool proposed in [5] to evaluate the security of permutation Π against differential attack. Since the block size of Π is vary large, i.e., 2048-bit, the size of model is too large to be solved in a feasible time when attacked round $r > 2$. Instead of searching for differential characteristics with best differential probability (DP), we bound on the number of active words to significantly simplify the searching space. Based on our searching strategy, a 5-round differential characteristic were at most found with probability of 2^{-1903} . The final results are shown in Table 3. Despite many bounds about the number of active words are tried, we could not find a valid 6-round differential characteristic.

Table 3: Differential distinguishers for round-reduced Π

Attacked rounds	1	2	3	4	5
Number of active words	8	22	38	60	98
$-\log_2 \text{DP}$	8	34	194	1263	1903

4.2 Differential-Linear Cryptanalysis

This subsection gives differential-linear distinguishers against round-reduced version of the permutation Π . To this end, a r -round permutation E is divided into three parts: $E = E_2 \circ E_m \circ E_1$, where a r_1 -round differential trail $\Delta_{in} \rightarrow \Delta_m$ is used in E_1 , a r_m -round differential-linear approximation $\Delta_m \rightarrow \Gamma_m$ is used in E_m , and a r_2 -round linear trail $\Gamma_m \rightarrow \Gamma_{out}$ is used in E_2 , such that $r = r_1 + r_m + r_2$. Let p denote the differential probability of the differential trail of E_1 , r denote the correlation of the differential-linear approximation of E_m , and q denote the correlation of the linear trail of E_2 . For the permutation E , a r -round differential-linear approximation $\Delta_{in} \rightarrow \Gamma_{out}$ with correlation $\text{cor} = prq^2$ can be obtained. Assume that the input and output spaces of E are both $\mathbb{F}_2^n$. If $\text{cor} \gg 2^{-n/2}$ is satisfied, one can use the above r -round differential-linear approximation to distinguish E from a pseudorandom permutation.

The search for differential-linear distinguishers adopts a typical hourglass structure. Concretely, for E_m we traverse all combinations of 1-bit input differences $\Delta_m = [i]$ with 1-bit or consecutive 2-bit output linear masks $\Gamma_m \in \{[j], [j, j+1]\}$; For each combination we extend Δ_m (resp. Γ_m) backward (resp. forward) and automatically search for the optimal

differential trail (resp. linear trail) in E_1 (resp. E_2). Setting $(r_1, r_m, r_2) = (1, 2, 2)$, the best 5-round differential-linear distinguisher we obtained has a correlation $2^{-32.83}$. For the configuration $(r_1, r_m, r_2) = (2, 2, 2)$, since the differential probability weight is large, the time cost of the search procedure is high. In order to construct a 6-round distinguisher, we extend the above 5-round distinguisher backward by 1 round in the differential part. Finally, we got a 6-round differential-linear distinguisher with correlation $2^{-176.83}$. Details of the 2 distinguishers are given in Table 4. It can be seen in Table 4 that when the round number of the differential trail in E_1 increases from 1 round to 2 rounds, the differential probability decreases from 2^{-8} to 2^{-152} . This shows the fast diffusion of Π 's round function.

Table 4: Differential-linear distinguishers for round-reduced Π

Round number r	(r_1, r_m, r_2)	(p, r, q)	(Δ_m, Γ_m)	Correlation
5	$(1, 2, 2)$	$(2^{-8}, 2^{-0.83}, 2^{-12})$	$([1343], [1975])$	$2^{-32.83}$
6	$(2, 2, 2)$	$(2^{-152}, 2^{-0.83}, 2^{-12})$	$([1343], [1975])$	$2^{-176.83}$

4.3 Integral Cryptanalysis

The concept of integral cryptanalysis was first formalized by Knudsen and Wagner [6]. The core idea of integral cryptanalysis is finding a multiset of inputs such that the sum of some fixed parts of the resulting outputs is constant. The multiset of inputs in bit-based integral analysis usually consists of inputs taking all possible combinations in ℓ input bits, whereas the remaining $n - \ell$ bits take a fixed value. In the context of permutation, the output can be represented a vectorial Boolean function of input variables. If the degree of Algebraic Normal Form (ANF) of some output bits is less than ℓ , then these output bits are sum to zero, which can distinguish a cryptographic permutation from a random permutation. Division Property (DP) is a generalized integral cryptanalysis which was first introduced by Todo [7] at Eurocrypt 2015, which was subsequently utilized to break the full MISTY1 cipher at CRYPTO 2015 [8]. Later, Xiang et al. pioneered the application of automated tools for searching bit-based division property [9]. Building upon this foundation, Sun et al. extended these automated searching methods to the bit-level cryptanalysis of ARX ciphers [10].

The hash function Eijen introduced in this paper use an ARX construction as the non-linear component in the underlying permutation. Hence, we use the SAT-based automatic searching tools proposed by Sun et al. [10] to evaluate the division property. After setting the i -th bit indexed by $\{0, 1, \dots, 2047\} \setminus \{64\}$ as active bits, and fixing the 64-th bit as constant, we found a 4-round distinguished trail where the sum for each of the output bits is zero. And the experiments reveal that it is practically impossible to construct a 5-round integral distinguisher.

4.4 Preimage Cryptanalysis

For Sponge-F construction, a more appropriate perspective is to view the final absorption step as a truncated Davies–Meyer (DM) hashing mode. Hence, we can use the enhanced Meet-in-the-Middle (MitM) techniques to evaluate the security of **Eijen** against preimage attack.

For the permutation Π of **Eijen**, any non-zero difference in a single input word could achieves full diffusion after 3 rounds. Specifically, the active columns in each round can be represented as $1 \rightarrow 3 \rightarrow 8$, where the total 8 columns after the **MixQuad** layer in the third round are fully diffused. Partial matching [11] utilizes any determined relations between two neutral sets in the states at the matching point. To ensure at least one-bit valid matching point, the number of rounds used for the partial matching is upper bounded by $5 = ((3 - 1) + (3 - 1) + 1)$. Initial structure (IS) [12] is a few consecutive starting steps, where the two independent chunks overlapped. To ensure at least one neutral bit outside of the IS, the number of rounds used for IS is also upper bounded by 3. The Splice-and-Cut [11] attack framework is very suitable for DM hashing mode, and allows an attacker to add more 3 rounds in the worst case. In summary, at most 11-round ($3 + 3 + 5$) MitM preimage attack may be feasible, leaving a security margin of $16 - 11 = 5$ rounds.

4.5 (Quantum) Collision Cryptanalysis

For hash functions, the critical issue is not merely whether high-probability differential paths exist within the underlying permutation, but rather whether these paths can be translated into collision-generating differentials within the full Sponge-F mode. As discussed in Section 4.1, at most a 5-round differential characteristic for permutation with probability of 2^{-1903} can be found, thus the full-round **Eijen** is secure enough against (quantum) collision attack and (Semi-) Free-Start collision attack.

4.6 Other Cryptanalysis

This subsection evaluates attack methods explicitly considered in the submission evaluation criteria. The analysis is method-oriented: for each attack class, we identify the attack target induced by the Sponge-F mode and then relate the expected complexity to the exposed digest length h , the capacity c , the number of padded message blocks ℓ , and, when applicable, the fixed output subset size t .

Joux-Style Multi-Collision Attacks. The classical Joux multi-collision method applies to Merkle–Damgård iterations by repeatedly finding two-block alternatives that collide to the same n -bit chaining value. Repeating this process for s stages yields a 2^s multi-collision with an expected cost of about $s \cdot 2^{n/2}$, rather than the much larger direct birthday cost for all 2^s messages at once.

Joux-style multi-collision attacks are applicable to the Sponge-F iteration at the capacity-chaining level. This is a concrete consequence of the Sponge-F parameter choice: in order to increase the absorption rate, the capacity c is intentionally shorter than the capacity that would typically be required by a standard Sponge construction targeting the same preimage and second-preimage levels. Therefore, Joux-style multi-collisions can be constructed on the c -bit capacity chaining value; the correct question is whether the resulting $c/2$ -bit work factor remains above the claimed security target. For Eijen, the relevant chaining width is c , not h . More specifically, if S_{i-1} denotes the state after message injection in an absorbing step, then the capacity part after the feed-forward update is

$$B_i = \text{right}_c(\Pi(S_{i-1})) \oplus \text{right}_c(S_{i-1}).$$

Thus, a Joux-style step must find two message-block choices leading to the same c -bit capacity chaining value,

$$B_i = B'_i,$$

not merely to the same final h -bit digest. The expected complexity is about $s \cdot 2^{c/2}$ for a 2^s multicollision. Thus, the multi-collision resistance of the Sponge-F iteration is governed directly by the selected capacity length. Since all recommended instances use $c = h + 64$, this cost is

$$s \cdot 2^{c/2} = s \cdot 2^{h/2+32},$$

which remains 32 bits above the claimed $h/2$ -bit collision-security target. Table 5 lists the single-stage capacity-collision cost for the recommended instances; a 2^s multi-collision multiplies the corresponding entry by approximately s . The table should therefore be read as an explicit evaluation of the constructible Joux attack, rather than as a claim that Sponge-F rules out multi-collisions.

Table 5: Joux-style capacity-collision cost for Eijen

Function	h	c	Single-Stage Cost
Eijen-256	256	320	2^{160}
Eijen-384	384	448	2^{224}
Eijen-512	512	576	2^{288}
Eijen-768	768	832	2^{416}
Eijen-1024	1024	1088	2^{544}

Length-Extension Attacks. In a Merkle–Damgård hash function, a length-extension attack uses the public digest as an exposed chaining value and continues the compression process on attacker-chosen suffix blocks. This method relies on the digest containing enough internal state information to resume hashing.

The Eijen digest is not an exposed chaining value. It is an h -bit truncation of the final capacity part, while the full 2048-bit state and the rate part remain unknown. The rate-side

pad_{10*1} rule fixes an unambiguous final message-block boundary, but this padding alone is not the basis of the length-extension claim. In addition, the recommended parameter sets reserve $c - h = 64$ capacity bits that are not included in the digest, and the final absorbing block injects the instance-specific D_h into the capacity part before applying the permutation and the capacity feed-forward. Thus an attacker who knows only $H_h(M)$ cannot reconstruct either the hidden rate or the complete capacity chaining value needed to continue absorbing a suffix.

5 Design Rationale

The Eijen design targets high-security hashing under next-generation commercial cryptography requirements. The design considers both security requirements and software and hardware implementation constraints, and it separates the mode-level structure from the underlying permutation. This layered presentation keeps the hash mode and permutation design relatively independent, which simplifies specification, implementation, and cryptanalysis.

5.1 Sponge-F Structure

Capacity Feed-Forward. A fundamental characteristic of the standard Sponge construction is that the underlying permutation Π is public and invertible. Therefore, given any b -bit intermediate state, an attacker can evaluate it simultaneously in both forward and backward directions, thereby allowing direct stitching between consecutive permutation calls. This invertibility serves as the core point of exploitation for preimage and second-preimage attacks on the standard Sponge. Classical attacks utilize Π^{-1} to trace backwards during the squeezing phase, and then perform a meet-in-the-middle match with the forward search in the absorbing phase [13].

Enhanced Security Bounds and Efficiency. Compared to the standard Sponge construction, the collision and (second) preimage security of Sponge-F can be reduced to simple and well-defined correlation intractability properties of the underlying permutation, which enables the establishment of precise and meaningful security bounds in both classical and quantum settings. Besides, compared to Sponge-DM [2], Sponge-F could reach identical security levels using a significantly smaller permutation state, thereby effectively compensating for the feed-forward overhead and making it highly suitable for constrained devices.

Large-State Digest Squeezing. The design requirements of China’s next-generation commercial hash functions specify 1024-bit digests and 1024-bit preimage and second preimage security. And standard sponge construction cannot achieve such a high level of security. While Sponge-F can provide an instance with $h = 1024$ and $c = 1088$, yielding 1024-bit preimage and 1024-bit second preimage security.

5.2 Design Choices for Permutation

With the goal of approximating a random permutation, we design a concrete permutation instance. We opt to instantiate it using a large 2048-bit state permutation. Such a substantial state size allows for higher absorption rates without significantly compressing the capacity. All configurations uniformly adhere to the capacity formula:

$$c = h + 64$$

where $64 = \lceil \log_2 \ell_{\max} \rceil$ ensures that the second-preimage security still achieves the full h bits even under the maximum message length of $2^{64} - 1$ blocks. For instance, in the 256-, 384-, 512-, and 768-bit configurations, the capacities are 320, 448, 576, and 832 bits, respectively, with corresponding rates of 1728, 1600, 1472, and 1216 bits. All of these maintain a large absorption width. For the 1024-bit configuration, where

$$(r, c) = (960, 1088)$$

although the rate is somewhat reduced, it remains sufficient to provide reasonable throughput. The permutation function employs an iterated round-function structure.

The design of the round function thoroughly accounts for strong diffusion properties and compatibility with CPU parallel instructions. The round function is defined as follows:

$$\mathcal{R} = \text{PosPerm} \circ \text{AMix4} \circ \text{PosPerm} \circ \text{MixQuad}.$$

The isolated **MixQuad** layer operates locally on a column-by-column basis; while convenient to implement, it is insufficient to provide global diffusion. The first positional permutation breaks this column locality, the column mixing layer amplifies the activity within each affected column, and the second positional permutation then redistributes the amplified activity across the full width of the state. This decomposition strategy aims to achieve rapid qualitative diffusion while preserving implementation regularity.

The design of **MixQuad** improves upon the QuarterRound (QR) of ChaCha by avoiding backward paths with a probability of 1. Furthermore, path probabilities and bit diffusion properties were comprehensively considered during parameter selection. Regarding the overall structural design, we chose to utilize two distinct sets of **MixQuad** parameters. This prevents the uniform repetition of eight completely isomorphic columns, thereby mitigating the probability of identical local structures emerging in the left and right halves of the state. Concurrently, in combination with the subsequent structural components, this design avoids differential cancellation and enhances overall diffusion. The selection of the two sets of rotation parameters adheres to the following design criteria:

1. **Value Constraints:** The six values π_0 – π_5 are mutually distinct and all fall within the range $[3, 62]$.
2. **Distance Constraints:** The cyclic distance between π_4 and π_0 is ≥ 3 ; the cyclic distance between π_5 and π_1 is ≥ 3 .

3. **Modular Addition Constraints:** $\pi_1 + \pi_2 \not\equiv 0$, $\pi_2 + \pi_3 \not\equiv 0$, $\pi_3 + \pi_5 \not\equiv 0$, and $\pi_0 + \pi_j \not\equiv 0$ ($j = 1, 2, 3, 5$), all $(\text{mod } 64)$.
4. **Algebraic Constraints:** $(\pi_1 + \pi_2) \text{ mod } 64 \in [20, 40]$, $(\pi_3 + \pi_2) \text{ mod } 64 \in [20, 40]$, $\pi_1 + \pi_2 \not\equiv \pi_5$, $\pi_0 + \pi_1 \not\equiv \pi_4$, $\pi_3 + \pi_5 \not\equiv \pi_1$, $\pi_3 + \pi_2 \not\equiv \pi_1$, and $\pi_0 + \pi_1 \not\equiv \pi_4 + \pi_3 + \pi_2$, all $(\text{mod } 64)$.
5. **Single-Bit Diffusion Optimization:** Through forward and inverse experimental evaluations of 256 types of single-word, single-bit differences, the parameter sets with the highest comprehensive scores are selected from the candidates satisfying the aforementioned constraints.

The parameter construction of the **PosPerm** permutation is designed to break column locality while accounting for software instruction support and diffusion properties. It ensures that the four words of a single column are scattered into distinct columns. Subsequently, through the **AMix4** operation, one active column influences 12 words, and a second positional permutation further diffuses this influence across all columns. That is,

$$\sigma = [22, 0, 29, 19, 26, 4, 1, 23, 30, 8, 5, 27, 2, 12, 9, 31, \\ 6, 16, 13, 3, 10, 20, 17, 7, 14, 24, 21, 11, 18, 28, 25, 15].$$

Let ρ denote the function computing the output position for an input word. Then,

$$\rho(4j + i) = 4((j + s_i) \text{ mod } 8) + t_i,$$

where

$$(s_0, s_1, s_2, s_3) = (0, 1, 3, 4), \quad (t_0, t_1, t_2, t_3) = (1, 2, 0, 3).$$

Consequently, the four words within a single input column are mapped to four distinct output columns:

$$\{j, j + 1, j + 3, j + 4\} \pmod{8}.$$

6 Features

The design requirements for hash algorithms within China's next-generation commercial cryptography framework are as follows:

1. The algorithm shall support at least two hash output lengths, 512 bits and 1024 bits, with the option to support a 768-bit output length. The maximum supported message length shall be at least $2^{64} - 1$ bits.
2. The algorithm shall be secure against known classical and quantum attacks.
3. The algorithm shall be efficiently implementable across a broad range of software and hardware platforms, ensuring high practical applicability.

The design of the proposed algorithm adheres to the following principles:

1. **Security:** The algorithm shall provide preimage resistance, second-preimage resistance, and collision resistance, thereby ensuring resistance against known cryptanalytic methods. The design also consider common attacks such as meet-in-the-middle (MitM) attacks and algebraic attacks.
2. **Implementation efficiency:** The hash algorithm can be executed efficiently across a wide range of software and hardware platforms. On software platforms, it is suitable for optimized 32-bit and 64-bit implementations; on hardware platforms, it take advantage of modular addition to achieve a highly favorable trade-off between compact circuit area and execution speed.
3. **Flexibility:** The algorithm is designed to adapt to different application requirements and operational environments. Subject to the stated security requirements, it supports multiple output digest lengths, such as 256, 384, 512, 768 and 1024 bits, for applications requiring different security levels.
4. **Algorithmic simplicity:** The implementation remains simple enough for precise specification, review, and deployment. Reducing implementation complexity lowers the risk of implementation errors and supports deployment in resource-constrained environments.

7 Implementation and Evaluation

We provide a portable reference implementation and an x86-64 AVX2 single-message implementation for all five digest lengths. Both families implement the same **CryptHash** bit-length interface, the revised **pad10*1** rule, little-endian serialization, and capacity feed-forward including D_h . The reference code is located in:

`Implementations/Reference_Implementation/`

The optimized code is located in:

`Implementations/Optimized_Implementation_AVX256_Single/`

7.1 Software Implementations

Reference implementation. The reference implementation stores the 2048-bit state as 32 64-bit words. It evaluates each **MixQuad** column in place using modular additions, XORs, and rotations. The **PosPerm** layer follows the cycle decomposition of σ , and **AMix4** uses the explicit four-word XOR formulas. The implementation processes arbitrary bit lengths and masks the unused high bits of a non-byte-aligned final input byte.

AVX2 single-message implementation. The optimized implementation targets one message at a time on x86-64 AVX2. Eight 256-bit YMM vectors hold the state in a row-oriented layout. The four 64-bit lanes of each vector contain words from four distinct MixQuad columns of the same message, rather than four independent messages. An initial pair of 4×4 transpositions establishes this layout, which remains in registers throughout all 16 rounds.

Packed additions and XORs use AVX2 integer instructions. Most rotations use two shifts and an OR, while the 56-bit rotation uses a byte shuffle. PosPerm is implemented with lane permutations, blends, and register reassignment, and AMix4 is evaluated with packed XORs. A single-rate-block fast path constructs the final padded state directly in the vector layout. Longer inputs are absorbed block by block without converting the state layout between rounds. This implementation exposes no four-message or AVX-512 interface.

7.2 Evaluation Methodology

Measurements were collected on an Intel Xeon Silver 4416+ processor (20 cores and 40 hardware threads) running Ubuntu 22.04.5. The toolchain was GCC 11.4.0 and CMake 3.22.1. The reference implementation was compiled with `-O2`; the AVX2 implementation used `-O3 -march=x86-64 -mavx2`. Both builds enabled C99 conformance and warnings as errors.

Each benchmark process was pinned to logical CPU 2. We measured messages of 32, 128, 512, 1024, 4096, 8192, 16384, and 65536 bytes. Each reported value is the median of 101 samples. Cycle counts were read with RDTSCP and a serialization fence, while throughput used CLOCK_MONOTONIC_RAW. The benchmark mutates one input byte after each call to prevent repeated-call elimination. Before each measured run, one complete benchmark pass per binary was used for warm-up and excluded from the reported results.

Before measurements, the five reference builds and five AVX2 builds passed their self-tests. The tests cover the abc known-answer value, the empty message, invalid API inputs, non-byte-aligned messages, and lengths around each rate boundary. The complete formal KAT files are provided in `Test_Vectors/`.

7.3 Performance Results

We measured all five reference and AVX2 variants from the current submitted sources under the conditions described above.

The optimized implementation required 12.15–12.46 cycles/Byte for a 32-byte input in four instances and 16.38 cycles/Byte for Eijen-1024. At 65536 bytes, it achieved 1.58–2.84 cycles/Byte and 704–1265 MB/s. Relative to the corresponding Eijen reference implementation, the long-message speedup is $2.19\times$ – $2.34\times$.

Table 6: Single-message performance of the current submitted implementation on the Intel Xeon Silver 4416+. Values are medians of 101 samples after one excluded warm-up pass per binary; throughput is decimal MB/s.

Instance	AVX2 32 B cycles/Byte	Reference 65536 B cycles/Byte	AVX2 65536 B cycles/Byte	AVX2 65536 B MB/s	Reference-to-AVX2 speedup
Eijen-256	12.456	3.464	1.581	1265.41	2.19×
Eijen-384	12.268	3.981	1.701	1175.73	2.34×
Eijen-512	12.151	4.059	1.855	1077.97	2.19×
Eijen-768	12.147	5.229	2.233	895.85	2.34×
Eijen-1024	16.375	6.650	2.841	703.88	2.34×

8 Randomness Tests

Since the five variants *Eijen-h* utilized the same permutation, we only evaluate the randomness about the full 2048-bit output of the permutation function Π . Three distinct types of inputs were tested:

- **Dense data:** The input messages consist of approximately 95% 1s and 5% 0s.
- **Sparse data:** The input messages consist of approximately 5% 1s and 95% 0s.
- **Random data:** The input messages are generated uniformly at random.

For each input category, 1000 input streams are sampled and each stream includes of 1,000,000-bit, which were subsequently subjected to randomness testing. The implementation of the randomness tests and the selection of testing parameters strictly adhere to industry standard specifications. The results are shown in Table 7.

References

- [1] Chun Guo, Kai Hu, Shuntian Jiang, Yanhong Fan, Yong Fu, Bart Preneel, and Meiqin Wang. Permutation-based hash from non-idealized assumptions: Adding feed-forward to sponge. Cryptology ePrint Archive, Paper 2025/1006, 2025.
- [2] Siwei Sun, Shun Li, Zhiyu Zhang, Charlotte Lefevre, Bart Mennink, Zhen Qin, and Dengguo Feng. Permutation-based hashing with stronger (second) preimage resistance. Cryptology ePrint Archive, Paper 2025/963, 2025.
- [3] National Institute of Standards and Technology. SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions. Technical Report FIPS PUB 202, National Institute of Standards and Technology, 2015.

Table 7: Randomness test results for the 2048-bit output of the permutation function Π (significance level $\alpha = 0.01$; pass threshold ≥ 981 out of 1000 samples)

Statistical Test	Dense data		Sparse data		Random data	
	Pass Rate	Result	Pass Rate	Result	Pass Rate	Result
Monobit Frequency	993/1000	Pass	988/1000	Pass	992/1000	Pass
Block Frequency m=10000	993/1000	Pass	989/1000	Pass	987/1000	Pass
Poker m=4	989/1000	Pass	996/1000	Pass	989/1000	Pass
Poker m=8	989/1000	Pass	993/1000	Pass	985/1000	Pass
Serial m=3, P1	993/1000	Pass	989/1000	Pass	993/1000	Pass
Serial m=3, P2	993/1000	Pass	986/1000	Pass	996/1000	Pass
Serial m=5, P1	991/1000	Pass	993/1000	Pass	994/1000	Pass
Serial m=5, P2	989/1000	Pass	995/1000	Pass	990/1000	Pass
Runs	990/1000	Pass	991/1000	Pass	993/1000	Pass
Runs Distribution	991/1000	Pass	988/1000	Pass	988/1000	Pass
Longest Run of Ones m=10000	992/1000	Pass	995/1000	Pass	989/1000	Pass
Longest Run of Zeros m=10000	989/1000	Pass	995/1000	Pass	986/1000	Pass
Binary Derivative k=3	987/1000	Pass	989/1000	Pass	990/1000	Pass
Binary Derivative k=7	992/1000	Pass	987/1000	Pass	992/1000	Pass
Autocorrelation d=1	990/1000	Pass	991/1000	Pass	993/1000	Pass
Autocorrelation d=2	993/1000	Pass	991/1000	Pass	994/1000	Pass
Autocorrelation d=8	989/1000	Pass	985/1000	Pass	989/1000	Pass
Autocorrelation d=16	991/1000	Pass	988/1000	Pass	990/1000	Pass
Binary Matrix Rank	992/1000	Pass	991/1000	Pass	990/1000	Pass
Cumulative Sums (Forward)	994/1000	Pass	988/1000	Pass	988/1000	Pass
Cumulative Sums (Backward)	992/1000	Pass	989/1000	Pass	991/1000	Pass
Approximate Entropy m=2	993/1000	Pass	989/1000	Pass	993/1000	Pass
Approximate Entropy m=5	996/1000	Pass	990/1000	Pass	991/1000	Pass
Linear Complexity m=500	990/1000	Pass	994/1000	Pass	992/1000	Pass
Linear Complexity m=1000	986/1000	Pass	995/1000	Pass	990/1000	Pass
Maurer's Universal L=7, Q=1280	986/1000	Pass	995/1000	Pass	988/1000	Pass
Discrete Fourier Transform	989/1000	Pass	987/1000	Pass	988/1000	Pass

- [4] Kai Fu, Meiqin Wang, Yinghua Guo, Siwei Sun, and Lei Hu. Milp-based automatic search algorithms for differential and linear trails for speck. In Thomas Peyrin, editor, *Fast Software Encryption - 23rd International Conference, FSE 2016, Bochum, Germany, March 20-23, 2016, Revised Selected Papers*, volume 9783 of *Lecture Notes in Computer Science*, pages 268–288. Springer, 2016.
- [5] Ling Sun, Wei Wang, and Meiqin Wang. More accurate differential properties of LED64 and midori64. *IACR Trans. Symmetric Cryptol.*, 2018(3):93–123, 2018.
- [6] Lars R. Knudsen and David A. Wagner. Integral cryptanalysis. In Joan Daemen and Vincent Rijmen, editors, *Fast Software Encryption, 9th International Workshop, FSE 2002, Leuven, Belgium, February 4-6, 2002, Revised Papers*, volume 2365 of *Lecture Notes in Computer Science*, pages 112–127. Springer, 2002.
- [7] Yosuke Todo. Structural evaluation by generalized integral property. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology - EUROCRYPT 2015 - 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26-30, 2015, Proceedings, Part I*, volume 9056 of *Lecture Notes in Computer Science*, pages 287–314. Springer, 2015.
- [8] Yosuke Todo. Integral cryptanalysis on full MISTY1. In Rosario Gennaro and Matthew Robshaw, editors, *Advances in Cryptology - CRYPTO 2015 - 35th Annual Cryptology Conference, Santa Barbara, CA, USA, August 16-20, 2015, Proceedings, Part I*, volume 9215 of *Lecture Notes in Computer Science*, pages 413–432. Springer, 2015.
- [9] Zejun Xiang, Wentao Zhang, Zhenzhen Bao, and Dongdai Lin. Applying MILP method to searching integral distinguishers based on division property for 6 lightweight block ciphers. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *Advances in Cryptology - ASIACRYPT 2016 - 22nd International Conference on the Theory and Application of Cryptology and Information Security, Hanoi, Vietnam, December 4-8, 2016, Proceedings, Part I*, volume 10031 of *Lecture Notes in Computer Science*, pages 648–678, 2016.
- [10] Ling Sun, Wei Wang, and Meiqin Wang. Automatic search of bit-based division property for ARX ciphers and word-based division property. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology - ASIACRYPT 2017 - 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part I*, volume 10624 of *Lecture Notes in Computer Science*, pages 128–157. Springer, 2017.
- [11] Kazumaro Aoki and Yu Sasaki. Preimage attacks on one-block md4, 63-step MD5 and more. In Roberto Maria Avanzi, Liam Keliher, and Francesco Sica, editors, *Selected Areas in Cryptography, 15th International Workshop, SAC 2008, Sackville, New Brunswick, Canada, August 14-15, Revised Selected Papers*, volume 5381 of *Lecture Notes in Computer Science*, pages 103–119. Springer, 2008.

- [12] Yu Sasaki and Kazumaro Aoki. Finding preimages in full MD5 faster than exhaustive search. In Antoine Joux, editor, *Advances in Cryptology - EUROCRYPT 2009, 28th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cologne, Germany, April 26-30, 2009. Proceedings*, volume 5479 of *Lecture Notes in Computer Science*, pages 134–152. Springer, 2009.
- [13] Xiaoyang Dong, Boxin Zhao, Lingyue Qin, Qingliang Hou, Shun Zhang, and Xiaoyun Wang. Generic mitm attack frameworks on sponge constructions. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part IV*, volume 14923 of *Lecture Notes in Computer Science*, pages 3–37. Springer, 2024.

A Round Constants

Table 8: Round constants used in MixQuad

Round	$ci_0^{(r)}$	$ci_1^{(r)}$
0	0x243F6A8885A308D3	0x13198A2E03707344
1	0xA4093822299F31D0	0x082EFA98EC4E6C89
2	0x452821E638D01377	0xBE5466CF34E90C6C
3	0xC0AC29B7C97C50DD	0x3F84D5B5B5470917
4	0x9216D5D98979FB1B	0xD1310BA698DFB5AC
5	0x2FFD72DBD01ADFB7	0xB8E1AFED6A267E96
6	0xBA7C9045F12C7F99	0x24A19947B3916CF7
7	0x0801F2E2858EFC16	0x636920D871574E69
8	0xA458FEA3F4933D7E	0x0D95748F728EB658
9	0x718BCD5882154AEE	0x7B54A41DC25A59B5
10	0x9C30D5392AF26013	0xC5D1B023286085F0
11	0xCA417918B8DB38EF	0x8E79DCB0603A180E
12	0x6C9E0E8BB01E8A3E	0xD71577C1BD314B27
13	0x78AF2FDA55605C60	0xE65525F3AA55AB94
14	0x5748986263E81440	0x55CA396A2AAB10B6
15	0xB4CC5C341141E8CE	0xA15486AF7C72E993

B Test Vectors

All messages, state words, and digests in this appendix use the normative little-endian convention. Message bits are consumed LSB-first within each byte. The all-zero and all-one messages are exactly r bits long; the string “abc” is 24 bits long. The vectors include rate-side pad_{10*1} and the instance-specific final capacity constant D_h , injected before the permutation and included in capacity feed-forward.

B.1 Test Vectors for Eijen-256

$r = 1728\text{-bit all-zero message.}$

0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000					

Final digest word values:

0615910A9A7BA7FA	CC58E804113DA162	BC5E97C5EC6E1A23	6AD927C5C552C182
------------------	------------------	------------------	------------------

Little-endian serialized digest bytes:

FAA77B9A0A911506	62A13D1104E858CC	231A6EECC5975EBC	82C152C5C527D96A
------------------	------------------	------------------	------------------

$r = 1728\text{-bit all-one message.}$

FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF					

Final digest word values:

F8CB1EF777F1FB83	95844F2E7032CE9A	488A683A08984201	87D65E1F967DE38E
------------------	------------------	------------------	------------------

Little-endian serialized digest bytes:

83FBF177F71ECBF8	9ACE32702E4F8495	014298083A688A48	8EE37D961F5ED687
------------------	------------------	------------------	------------------

24-bit input string “abc” (ASCII 616263).

Final digest word values:

832DA07964E1156C	F0EE3CC38C0CED82	3D6C9CD0ED919514	E7C9C4D1DD6DD3ED
------------------	------------------	------------------	------------------

Little-endian serialized digest bytes:

6C15E16479A02D83	82ED0C8CC33CEEFO	149591EDD09C6C3D	EDD36DDDD1C4C9E7
------------------	------------------	------------------	------------------

Padded rate block for “abc”. The first word is 0000000001636261; the last rate word is 8000000000000000. The capacity-side D_{256} is injected independently as 8000000000000100 in the final state word.

0000000001636261	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	8000000000000000					

Eijen-256 “abc” round trace. S_0 is the state after injecting the padded rate block and D_{256} . Rows R_1 through R_{16} are the states after each permutation round. X_1 is the state after the capacity feed-forward; its rightmost 256 bits form the digest.

State	Eight consecutive 64-bit words per row							
S_0	0000000001636261	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
	0000000000000000	0000000000000000	8000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	8000000000000100
R_1	13198A2E03707344	BB434A9F5E957D85	9E1DD29622F25502	6308A8B848691D25	1319AA2E03707344	C818A83C3E9FA93A	4E0370734532A94A	5C009EDAAA327A7A
	A30768AB65A1BFEC	E430EAF3A86BAA1A	3D2F04DE30BC7DF5	5C00BEDAAA327A7A	2444FC8E902708D3	E470CAEBB06BAA1A	11436A09BE487ED5	EC1E7C5FCCD336F2
	24336A8085A308D3	E308AA71FE6BDF95	8D29262D6E9B70F9	BB434A9F5E957D85	247746909DA308D3	5C009EDAAA327A7A	910B44D8104ABC65	C814A8343E9FA93A
	B81DOAB44D7006FF	5C00BEDAAA327A7A	2E0370734413198A	E478C6EBB06BAA1A	931988E7B572B1F4	5C00DEBAAA02FA5A	2E0370734413198A	7852AAD778B8A436
R_2	8639CE99B76B521A	AC40F7D389D737BE	7BD4CBE27965755E	F20F036816507EBC	A44534068AD6AC1D	FD9E9F657A672FAD	1D19AADCA124272F	37125103987950A8
	9A1EB13609269A5D	6563B448220B1D17	8BF64D532D9E3365	47D6D5DC75C76823	282B1260A5DA4500	3C061B8ECBF8BA68	A60B832ABBCE2594	1C2C5C48C9C57642
	D065ACA8D6DE3A02	900775369476088D	1C6D6552EC79AA00	EFD77987C6A226C5	D9EA6835186CFA22	739986D3ECF06201	18EBFBACAD0364B0	6BBD0F0DD6FF18FF
	287A4081E3D65A7D	891EDAD33D4D097F	AD6B90CA500A3136	4347982C2DFFD63A	4689B734712AA8C1	BA5CD2A2E8D47537	F17D59525FE831E0	93D0DE4DB33780F8
R_3	03998221CA83B66E	DED5F62D99111916	74C29F3A1D07818A	21A5A9F87D92CD34	9EB3DBDDFEF1D13A	5D38BB2F5DEA4CB8	AB77F80999CC7D93	B2A55DAB5A9C70D0
	C547CBE82777EF2C	32D59A6DA92A6EE2	87C0307049C15B18	3FD1F7628DA9351F	3B3647FD1C278198	E5A63CF96BEFA877	6FD93074EB2BBE3C	E02AD5EB202560FC
	ABF05B7DDDE3A185	B8135FBEC873F883	1F7CA6CE5CD6A848	3A0EC7C91BC5B8D6	1EC49E052BC98EE6	2E5A6A4DB614F102	D40EE60B8EA1FD58	9F13BC0270145C6D
	2BFF8EB47063207E	4740839B2B2CD99F	9F36DFDE01D873CC	D4090B760448C602	821CC0C6D06D8B6EE	B1AC2549F5394009	BC773AA86A91A8D9	C2053AA3EF239339
R_4	74C9D660BEF4BFD	1082DD07C83B0AEC	AEAC4573FE5D28A8	DF6C623523896E8B	B72FF3970FBC98FC	E4C01D53C7643A5A	4EB8DFA72C7C2586	0EF9634B35BD1127
	56D1D19A46663DCD	0028B88D0CCCB34	0F29847674B4BBC6	A63A98B3265F49B8	2678522DA582406D	665BCE99CDDCE51D	B5D4FBEB50DC8B34	B3E7AB8B5C1B16CD
	6D1A4128D3F8B591	8280066D88499785	652AB23805692A13	AA0E2A9E7FD94840	71C23D3616DA7614	30828C1F7B0852C3	AB4655DC525FE491	E6B7B01F35C82F82
	E544809270A2648C	F74B631639644668	450E7B12A234FBA6	043C91A6B3B55588	8AF06C234BC176CD	FC3779FA51C012FD	111F242D2278E439	68A4AEFF8EDBE21C
R_5	C9FB116D6938F082	69C8B3D75B25C997	229AD7E9D1EF3417	033253AD1E048537	56076C86E9175623	67A6C43270543C43	64DE0B2FABFDE793	188CB0B5A1B90AF7
	1F8ABC63A1B2E35E	3F0420925CF1F3C8	432EC593DE26BD71	B534D884A8BA4FD5	7793A353E7DA0058	A55E571498C507C9	9CB738EFE2F1FC52	9E5A9CA73C774D8B
	DE3193895CF3C560	AAE5059C8EBB20E9	FEA8A7D1C9F5EEF9	65343EFF7109CB99	5B0D97BB2D3B3E8B	627BF16B87349B78	23ECDDBC3D7A227	E8F3C91A85B5CB97
	AC580B2EA15FE541	65D41D3CA370EDEB	0A95AF11F93B74CA	2CEC9335E255720B	EBFC6E0090254F31	ECD00C7DD3E73330	E112927D8F97C83B	C2F8759EE96CD849
R_6	D43802D21F2F7CC	F1B4F763E2F7CC72	7ECD1BC952484E0F	01C536ABCBC0C321D	139D52953AEOCBFD	BC8ECCB71FDBDEFC	6267A83D05EB2415	C33FCCC571B7268E
	2D394474E9E02406	8CBD2EC95FD27577	48008485938673F1	72407DB98DBD8CA0	73B884E6B359562E	5B9C62DF6888A549	17FE359B558DD87	C25707E313BA88C4
	5AB1B47CD4D55927	3846C16383DCFF3E	07E8E656C8C551C	81DA92BA2EC2064E	B5D664C75F5901BA	7CA38D35717929BE	E0160193A5143CBF	13420702736D425F
	3382FD06E1D62A35	2046B140A75E2A46	D032E0EDF08C2B6E	16BA456B9C3F8784	6601343E84B34D8F	1E924A9F51BE4D27	A5506A128DB58559	0F8D95C09A9A88B7
R_7	CD A8D0F0FF64A5E	60B3448269B378DA	F68C1E59DA1E1A67	4387C6D2A79E145F	B95F87955FBEEAA3	6AC22EA13965B873	5842985FFDC9A4A8	865B0E9505E7E41F
	0A9EC6F642BB788B	3372D294FB7EE779	603CEAB366B28A99	F46648AD774E6BC2	F5BD9502CCA4F4B6	F601290094AA885F	0BA15A7B9140563D	062F39103A6A2136
	2A58625EFD6E2039	0E528999BB998F49	81B404FE95C8C364	AC12755BE6360C8C	A26B2E40BDC7D89E	2B4BD719E0F95DAA	5D140C23ED979C09	1EA3AA9FE51EE443
	75CD27AF8CD0B780	50E6038CE947E7A8	EE707A393B2D35F1	2C1F3516F927DA6D	57E110CF3ED2F87D	EE33A73BAEF6F5DA6	28075BA7447FCD94	FADA83CE2A14B584
R_8	59BE3B9454F5DA3F	62C84FF8DEDA856	5373F925FC6D56D4	E2EB9652C884A953	58D9D7B20727E4A1	65BBCE7AFB82A7B4	B3A224848BB1C193	1A0B75317E739C83
	97FD5879D8D1F915	6E981366780340FE	D45CB92DD32638BA	DBC015856AC5963E	E6806CB682539F7	32623DB1F4CD6F8D	E0BE1F06FFACE6A8	BD082B4C0E84B972
	7791551BEC158117	D325AFD65051F20F	A6EA1C85916E90D5	F07321B728DA4ED0	94F0BA41D2D1678E	90EF69182B85FE77	58CDB4D19C5E2D30	6745B4CCB70A5616
	09D1A2A3397BCE67	9C2BEB7CB6A2DB1	1600C5AD1496D4AE	B556A4FF868DFA5E	9745A9EF0292F74C	EB47E07D7256E8F1	9D1B77C1659B5E34	80ECEA7E2D2FAE95
R_9	688D9F54A0CTA332	E393E742E8BAF126	637FEE74755B2AFC	20D78293F08DE719	14F60473B8BDD43C	118E3CD522756593	46BC1FE97F1F866F	2DD1602943DF41F
	0A10FE08C4B47654	01769E377843CB87	817CE55F2091D23C	835DD55778EF694F	8F6414E04BBBD567	D34B7124845F2435	9106E83D6C03AC7	F38FC90E509A0AE0
	2B1711023A02050F	441235115C7E79D8	3552C61DCAD374D5	C731220238946C8A	92C1FF8C8FF5A001	C1360C223B7988F2	6B7AEA1642A94D25	423F3B9EB2A62EA1

State	Eight consecutive 64-bit words per row															
R_{10}	92A7334EC348A8F9	16079A8BF964D17F	2644C5D078DA57A7	9B17DC690FE52D04	BDCE5CD7936F9B55	AB8C88519597DF28	96EBABCD0E38E0E3	A7E46EC405B98995								
	C7221B26A98646AE	AF5D6AF38DE35E40	771EA3BCC62A8E36	E98E1CA291AC42C6	951865A1D38DD27D	4A404DAF2729974A	BE8350FC3AC8D592	FB97BCD4CF6B3708								
	12D44BEC2B713470	2296316BF3D5209B	C4883EA28B39990E	91CA206ACBCC7069	5AD7560BD833F107	12DF45EDEFE820741	551CB6C51AAB81E1	F78C6B8B8E7CEA47								
	EDAF812BC4D15640	A36D1EE90300AD03	8762284BC11538AD	F091D87C100DF589	ED0FC32B0890E3CC	F16CF96153F4A92D	46C727C77BFBS2AF	B35DE7C09717686C								
	EEFFC961C09EF818	6DF3EB5D12F83029	6DF6BECC24BA77A0	C1E5D77552EAC10D	02FB0B932E9AEFB	070581546F3EBFC9	8A8E471D14FA46D5	BC0A86D1163198A								
R_{11}	0A4FFF1E896DECD	A48E4E2CE0B32BB	557E77BD7BEE864	55C5B78401F89C0F	A367F9E143CD9B0B	C4D2578F41B03D0A	E3C4B1F9F032F217	C9608F4A40532A9C								
	B157806FC81C08DE	8E527D32F3FBC145	54CF8CF4720E9FF5	C5D8F44660C2D481	4833D88AB794A48E	40D7BAE3F001EE46	BC2170BC9C71D23A	D1F3EC7D843C5330								
	1155B884BC337FCC	95784723B6CCFBB3	46D11C2C2610465C	06102CD29F0275F3	8F7449136BE672E6	77CDAF31B2819AA7	E89BFDDC72F9A8EF	2CCF2D4B63D60341								
	16CE766B2304395C	37306FD559085497	CABAC4BE9E3A2C29	907AA36870B5C3EE	F4EA3BF96A73306C	0D50F4E29F2608DA	203B8D0BC2E9BBC9	D30964D633AD9BF6								
R_{12}	DDBDFO00588A35C4	592COD6020B25E4D	7C22DE6FE7B14D4C	E707413C8A67C9BE	AC56D39A512B3FB1	FE9386E0EE18356F	F7D968A5F9275F82	0FD9C997FE6CAC18								
	C7986532DBDA8E46	0514F5CACAFA8735	F0CF130D77AD11CF	2E24D86D9488AFC4	D63EA1D720C16543	D4115DF1D40B70CD	BA4E5DF3B0C4E14B	079B04FFF53AAF64								
	326D46F60053F744	0A754D4B69A5E813	8C9B4DA074EC93E0	55F850252694E55E	1FC7E3C544F230D7	D903D52533D4B722	1BF76154E5BF7C35	0EFEC545F1146E6B								
	E607959CD1CA53F7	EA807F49DCF58FB9	4D43BB388EE7A3B8	E465B005981AC865	4D429A41C8BC1240	46C5665D92197F7A	1AEFEAAF645A0628	E0BFA5BD69C139F6								
R_{13}	E3213FCCF156A259	B93E1D94835AC317	2758578C85551164	33933E656842B185	27122202E413C319	24583E8C820EC229	F818BA4593A7ABDD	8AA0F7C12D8A1D45								
	EE753E95C4FEF87E	223B5C8290934EED	206E7E3226328661	86C5AEC EA7AC1A80	19DF2BACC2DD3E37	E15787021A568CB1	7CBD5F3CA81A0DA6	A8E35B9DFE8819C7								
	6E31564007D35921	86DFF772E3C34BBF	4CDF6E9A0D359FD9	3E3B119EA2242259	DC2B33284E8429DC	E72C07FFCF20C83D	37A57423073D5672	87996C299D1924C9								
	A4A0A23F8B43E5A1	C02138B0952322B8	B42C83D28C9B831B	7F3DEAD8485B0F4B	5A2C563C1B556263	ABCC4CDD9B9ED135	6F1C2E2CB4E1767B	74B663DE8D27CD69								
R_{14}	A0FC40D7300C1B4A	24B1EDF6D79FC4A5	364AAC86710F3842	28FCDF833CFC6F8C	ED0D825A0C61E569	DCB5F9B7691F8470	CFC6D8CDA27F5C8F	68E511DC80558577								
	3D460A19E7976A7E	2C1BA57381A3AD95	0A6576FCAA7468FA	ODA79860224E31F0	39BDC957BA11C688	85E41EF048EF572E	8FA3BB2E3D4D8E5D	6ECF975E013B78A0								
	665912A817D973ED	9C219D27CF41BF0B	FD7BE57E2841F34A	A4B2CCAFF0EC7F50	CAEAD608A9718B02	3E2A100E7BA4BD29	3361C830A125DA75	0E549ECD8A184BF0								
	0B5AFD2C983FC2FF	B990B25C78E4C13B	67C7711F751D7F2D	82D966284BC142DA	26E7DBC71A8EE94A	B6E25865640DACEF	53B978FFCDF864DA	42D445861B13E994								
R_{15}	7940C74B2AB6BA4D	F59E4B47ABB0CD4F	BC55A9C7DE05178B	ODAF3C341AFA1C26	02EA37EA70536A17	7A32710F71E96FE2	6432CBECEB4DA789	503060BC276675B4								
	5D26C3A4C7FD577F	34C01E3973E4E106	283D2A558BAB81BE	A354B76A46485C66	0896EE1D5507081E	F5463C7EE4809C85	F749679E9922794F	0C84F5A89CFEE553								
	D7472B5D9E38CA59	BAAD1B224A443C8A	F526CBDC222FCE8D	A5ACC6BE178CAE0B	56A5D51075FD3454	D3B7FA1E3FCD7EF	C342EECC7FDDCE8D	43BCEC2CAB171407								
	A14925BB859601AA	F34A6C3579F1A5E3	A75B4B25FAE9E776	E21D6E53121A5B8A	F61EC855D922B585	5B754D711428B7D7	EFA1DD3A4ECB44F8	4F39262FCEECBC7E								
R_{16}	DD1CFB59194B4DA9	474AD0A5BD21775E	4C4825A917B90E50	4987B04E8D5F5C7A	62B0C590DB6DC801	1B861A3E9C834A4D	266C22A59388068C	3E6F4B7C4D4C1771								
	173197256C772675	95741833EB45C633	F65B1C67EC8BD942	3978B0ED4E67A551	79F889F1DFAFDDAA	A050BBFC144648D1	425FBFDD8C872432	41F4D4476823E5D3								
	2981C0D647B2EEB2	5B718F7332F058DD	3F4368BC415523F9	4611A8A1FB115684	1E07C7B9A66171C7	D4006DD8BFEF71D1	8471B9251F5B3863	2B1B0A5739E9BD72								
	E272DE34F18B35CA	ED3FA484B8A8EB88	9AA6BF1764299A7A	BFF1E31CD2EC4BAC	832DA07964E1156C	F0EE3CC38C0CED82	3D6C9C0D0E919514	67C9C4D1DD6DD2ED								
X_1	DD1CFB59194B4DA9	474AD0A5BD21775E	4C4825A917B90E50	4987B04E8D5F5C7A	62B0C590DB6DC801	1B861A3E9C834A4D	266C22A59388068C	3E6F4B7C4D4C1771								
	173197256C772675	95741833EB45C633	F65B1C67EC8BD942	3978B0ED4E67A551	79F889F1DFAFDDAA	A050BBFC144648D1	425FBFDD8C872432	41F4D4476823E5D3								
	2981C0D647B2EEB2	5B718F7332F058DD	3F4368BC415523F9	4611A8A1FB115684	1E07C7B9A66171C7	D4006DD8BFEF71D1	8471B9251F5B3863	2B1B0A5739E9BD72								
	E272DE34F18B35CA	ED3FA484B8A8EB88	9AA6BF1764299A7A	BFF1E31CD2EC4BAC	832DA07964E1156C	F0EE3CC38C0CED82	3D6C9C0D0E919514	E7C9C4D1DD6DD3ED								

B.2 Test Vectors for Eijen-384

$r = 1600$ -bit all-zero message.

```
0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000
```

Final digest word values:

```
A326EEC27A475DC8 C7E8369CC2B4C727 2F8B1FD3C96F02B2 18FA56D2FDC2476E 91C81CA34AE7B8D0 841173D5AD7B5377
```

Little-endian serialized digest bytes:

```
C85D477AC2EE26A3 27C7B4C29C36E8C7 B2026FC9D31F8B2F 6E47C2FDD256FA18 D0B8E74AA31CC891 77537BADD5731184
```

$r = 1600$ -bit all-one message.

```
FFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF
```

Final digest word values:

```
64DCB82A7A789CEB AFD3E7667743F3A5 B77D00C954377E89 27C56030283629F4 78AF8FD91A781F88 653FD58BE50E9CF1
```

Little-endian serialized digest bytes:

```
EB9C787A2AB8DC64 A5F3437766E7D3AF 897E3754C9007DB7 F42936283060C527 881F781AD98FAF78 F19C0EE58BD53F65
```

24-bit input string “abc” (ASCII 616263).

Final digest word values:

```
BFD2796EB2659988 F901109094B8A3D2 108B28AEE6934CFB 08339F99D3218E9A 4B2F3F00B44CAF78 BFD3E837441C5E37
```

Little-endian serialized digest bytes:

```
889965B26E79D2BF D2A3B894901001F9 FB4C93E6AE288B10 9A8E21D3999F3308 78AF4CB4003F2F4B 375E1C4437E8D3BF
```

B.3 Test Vectors for Eijen-512

$r = 1472$ -bit all-zero message.

0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000

Final digest word values:

BA60FC5F5996FAE7	9286C13226924127	A37D14AA83128E8A	95C365927444E3DC	F8EE663C3AAC2095	502EBE0BDBE1AF7E	A7D0CD700E3734C1	A690068D57D0C114
------------------	------------------	------------------	------------------	------------------	------------------	------------------	------------------

Little-endian serialized digest bytes:

E7FA96595FFC60BA	2741922632C18692	8A8E1283AA147DA3	DCE344749265C395	9520AC3A3C66EEF8	7EAFE1DB0BBE2E50	C134370E70CDD0A7	14C1D0578D0690A6
------------------	------------------	------------------	------------------	------------------	------------------	------------------	------------------

$r = 1472$ -bit all-one message.

FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF

Final digest word values:

9D90EBD51ADE37C5	DCA2A1E9D640A1A3	94EB67013FB95A93	82C5B8A4632B9378	3133FB9C19550CDA	71176D4041C4101A	EB91C623170D06FC	668495F85BFD9503
------------------	------------------	------------------	------------------	------------------	------------------	------------------	------------------

Little-endian serialized digest bytes:

C537DE1AD5EB909D	A3A140D6E9A1A2DC	935AB93F0167EB94	78932B63A4B8C582	DA0C55199CFB3331	1A10C441406D1771	FC060D1723C691EB	0395FD5BF8958466
------------------	------------------	------------------	------------------	------------------	------------------	------------------	------------------

24-bit input string “abc” (ASCII 616263).

Final digest word values:

89E634B57C1F62F1	0F77811BECA8BC2B	4DF27A6B378367F7	F5F1E69A69096F51	80E9300B724969D3	D32A6518FEE9DD02	60AEE785EF774BB2	4506C98D228BCFC0
------------------	------------------	------------------	------------------	------------------	------------------	------------------	------------------

Little-endian serialized digest bytes:

F1621F7CB534E689	2BBCA8EC1B81770F	F76783376B7AF24D	516F09699AE6F1F5	D36949720B30E980	02DDE9FE18652AD3	B24B77EF85E7AE60	C0CF8B228DC90645
------------------	------------------	------------------	------------------	------------------	------------------	------------------	------------------

B.4 Test Vectors for Eijen-768

$r = 1216$ -bit all-zero message.

0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000					

Final digest word values:

BB02C10509433C98	C09AD08EE897A88C	062F23F47EEA2F13	58AEC5FC3EA971B5	75A5201B27C51D25	AACFEA5B4472BFDA	78CDC82DB49A51DA	3F5BF96E2E4845C3
AEFC57D5AB2413AB	A582A304C37D83DB	BF49DF60D9420352	D985F1918DFD1EFE				

Little-endian serialized digest bytes:

983C430905C102BB	8CA897E88ED09AC0	132FEA7EF4232F06	B571A93EFCC5AE58	251DC5271B20A575	DABF72445BEACFAA	DA519AB42DC8CD78	C345482E6EF95B3F
AB1324ABD557FCAE	DB837DC304A382A5	520342D960DF49BF	FE1EFD8D91F185D9				

$r = 1216$ -bit all-one message.

FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF					

Final digest word values:

BF4E6A63E56F5D69	5CE499B319BE69F1	A47BE8F1C99EDF6E	A2050C55EBF10782	DC93592595C0C270	906BE9B14EC57421	6847BB55702B9DA4	4B48CCE415B6C660
1E14A8B42F191DE2	2BAE1D2993D5A9DE	DFE7A70ADB795208	C175923A78739F24				

Little-endian serialized digest bytes:

695D6FE5636A4EBF	F169BE19B399E45C	6EDF9EC9F1E87BA4	8207F1EB550C05A2	70C2C095255993DC	2174C54EB1E96B90	A49D2B7055BB4768	60C6B615E4CC484B
E21D192FB4A8141E	DEA9D593291DAE2B	085279DB0AA7E7DF	249F73783A9275C1				

24-bit input string “abc” (ASCII 616263).

Final digest word values:

FADC30ABA466EFEC	4B7268E45ECAD12B	61221E75E313F823	56451520B09F0BA4	9E54A2A4B3A465AF	0E063DD866F9A949	D574AD44B4902FDC	EE0DE34E84A030D2
E832188D26CBA6D4	CEC6ECB37DDE9D90	0D9DB23ED495DC41	637BB5D49CFB3BF7				

Little-endian serialized digest bytes:

ECEF66A4AB30DCFA	2BD1CA5EE468724B	23F813E3751E2261	A40B9FB020154556	AF65A4B3A4A2549E	49A9F966D83D060E	DC2F90B444AD74D5	D230A0844EE30DEE
D4A6CB268D1832E8	909DDE7DB3ECC6CE	41DC95D43EB29D0D	F73BFB9CD4B57B63				

B.5 Test Vectors for Eijen-1024

$r = 960$ -bit all-zero message.

0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
Final digest word values:							
7040B030CE5F2798	990E6D5675CCA816	322D3FF2B0A0389D	A4113AC07CF946B9	1786433DBBE33D93	DED4F3076243272C	EA141DC5DE4BCDCC	5B8A3601B51B196A
BF5232817681669C	E5F656B9543BA268	AB8A96C66E5B4A08	AE6DB8178C95A911	3452DBAD3BDE6CA7	665D84F67D0CA0CB	11B9B3547B50BFE1	76659AB778FD5E52
Little-endian serialized digest bytes:							
98275FCE30B04070	16A8CC75566D0E99	9D38A0B0F23F2D32	B946F97CC03A11A4	933DE3BB3D438617	2C27436207F3D4DE	CCCD4BDEC51D14EA	6A191BB501368A5B
9C668176813252BF	68A23B54B956F6E5	084A5B6EC6968AAB	11A9958C17B86DAE	A76CDE3BADD5234	CBA00C7DF6845D66	E1BF507B54B3B911	525EFD78B79A6576

$r = 960$ -bit all-one message.

FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF
Final digest word values:							
447F9395BB66ED71	CAF16F40186578BE	A8FC028F8383DD02	78284B5FCF079629	7C0E847DD501D02F	C3D188922ACFAF22	6F1F4C4B2BB1D9E5	00F8B6F1F4C1865F
DB979C2589EF6518	C870D6B1506880C2	E05E2B43093142AF	A9B7A00368208155	07341D31CF8569BD	1E3DF67D471128E5	B5E7808E36DD389D	79E2E0F1BC055C45
Little-endian serialized digest bytes:							
71ED66BB95937F44	BE786518406FF1CA	02DD83838F02FCA8	299607CF5F4B2878	2FD001D57D840E7C	22AFCF2A9288D1C3	E5D9B12B4B4C1F6F	5F86C1F4F1B6F800
1865EF89259C97DB	C2806850B1D670C8	AF423109432B5BE0	5581206803A0B7A9	BD6985CF311D3407	E52811477DF63D1E	9D38DD368E80E7B5	455C05BCF1E0E279

24-bit input string “abc” (ASCII 616263).

Final digest word values:							
A51B9B8903DF3936	ACECA089161027AF	7EC773E98D4BB265	4DC0675D93798706	A492CE7AC15DEC8B	6B5590AA95E4C1F8	C8BD91DEED2B4B0A	990A618A410D2BEB
B051703CFA79B994	EEA635918D775B37	B10A56BC88D1E1FB	2526CA24492634CD	3B4F9EC7144FD88F	2DB7946DD75EAE85	9174FF4F7695513A	91E75B62AA949293
Little-endian serialized digest bytes:							
3639DF03899B1BA5	AF27101689A0ECAC	65B24B8DE973C77E	068779935D67C04D	8BEC5DC17ACE92A4	F8C1E495AA90556B	0A4B2BEDDE91BDC8	EB2B0D418A610A99
94B979FA3C7051B0	375B778D9135A6EE	FBE1D188BC560AB1	CD34264924CA2625	8FD84F14C79E4F3B	85AE5ED76D94B72D	3A5195764FFF7491	939294AA625BE791